

Reviewer Pack — Minimal Monodromy Group Order and Communication Complexity R...

Entry 77cec2684e71 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 09:27:57 UTC

Minimal Monodromy Group Order and Communication Complexity Rank Inequality

Entry ID: 77cec2684e71 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 09:27:57 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Monodromy Theory) **Field B** (complexity object): Communication Complexity (Matrix Rank)

Statement:

For every n -vertex communication graph G , the order of the minimal monodromy group M_G acting on its incidence matrix is at most a polynomial in n with exponent $O(1)$, such that $|M_G| \leq \alpha(n)$ where $\alpha(n) = \Theta(\log^2(n))$. This implies an upper bound on the communication complexity rank $r(G)$ of G , i.e., $r(G) \leq O(\alpha(n))$.

Rationale (proposer's reasoning):

Monodromy theory studies the behavior of solutions under parameter variations, which has potential connections to the combinatorial structure of communication graphs. A lower bound on the monodromy group order could suggest a deep connection between algebraic structures and the complexity of communication tasks, providing insights into both fields.

Taxonomy category: AlgebraicGeometry × CommunicationComplexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 2f336080a9248446

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a given n -vertex graph G , we consider the minimal monodromy group M_G acting on its incidence matrix I_G . The criterion is met if $|M_G| \leq \alpha(n)$ with $\alpha(n) = \Theta(\log^2(n))$ and $r(G) \leq O(\alpha(n))$, where $r(G)$ is the communication complexity rank of G .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal monodromy group order" AND "communication complexity rank" - "incidence matrix" AND "monodromy theory" AND communication complexity- "polynomial bound" ON monodromy group ORDER BY communication complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_graph(n):
        edges = set()
        for i in range(n):
            for j in range(i + 1, n):
                if random.choice([True, False]):
                    edges.add((i, j))
        return {i: sorted(j for j in edges if j > i) for i in range(n)}

    def matrix_multiply(A, B):
        m = len(A)
        p = len(B[0])
        q = len(B)
        C = [[0] * p for _ in range(m)]
        for i in range(m):
            for j in range(p):
                for k in range(q):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def gaussian_elimination(A, b):
        m = len(A)
        n = len(b)
        Augmented = [A[i] + [b[i]] for i in range(m)]
        for i in range(m):
            max_row = i
            for j in range(i+1, m):
                if abs(Augmented[j][i]) > abs(Augmented[max_row][i]):
```

```

        max_row = j
        Augmented[i], Augmented[max_row] = Augmented[max_row], Augmented[i]
        pivot = Augmented[i][i]
        for j in range(n + 1):
            Augmented[i][j] /= pivot
        for j in range(m):
            if j != i:
                factor = Augmented[j][i]
                for k in range(n + 1):
                    Augmented[j][k] -= factor * Augmented[i][k]
    return [row[-1] for row in Augmented]

def communication_complexity_rank(G):
    n = len(G)
    I_G = [[0 if j != i else 1 for j in range(n)] + G[i] for i in range(n)]
    A = []
    for i in range(n):
        row = [I_G[j][i] for j in range(n, 2*n)]
        A.append(row)
    b = [1] * n
    x = gaussian_elimination(A, b)
    return sum(x[i] != 0 for i in range(n))

def minimal_monodromy_group_order(G):
    n = len(G)
    I_G = [[0 if j != i else 1 for j in range(n)] + G[i] for i in range(n)]
    S_n = list(itertools.permutations(range(n)))
    stabilizer_subgroup = []
    for sigma in S_n:
        permuted_I_G = [I_G[sigma[i]] for i in range(n)]
        if matrix_multiply(permuted_I_G, I_G) == I_G and matrix_multiply(I_G, permuted_I_G) == I_G:
            stabilizer_subgroup.append(sigma)
    return len(stabilizer_subgroup)

def alpha(n):
    return math.log2(n) ** 2

n_values = [5, 10, 15, 20, 30, 40]
metrics = []
for n in n_values:
    G = generate_random_graph(n)
    order_M_G = minimal_monodromy_group_order(G)
    r_G = communication_complexity_rank(G)
    metrics.append({
        "n": n,
        "order_M_G": order_M_G,
        "r_G": r_G
    })

mean_metric_value = sum(metric["order_M_G"] for metric in metrics) / len(metrics)
std_metric_value = math.sqrt(sum((metric["order_M_G"] - mean_metric_value) ** 2 for metric in metrics) / len(metrics))
conjecture_holds = all(metric["order_M_G"] <= alpha(metric["n"]) and metric["r_G"] <= alpha(metric["n"]) for metric in metrics)
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "minimal_monodromy_group_order",

```

```

        "metric_value": mean_metric_value,
        "instances_tested": len(metrics),
        "n_max": max(metric["n"] for metric in metrics),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(sys.argv[1])] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value) ** 2 for result
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7cd11187.py", line 116, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7cd11187.py", line 88, in run_trial
    G = generate_random_graph(n)
        ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7cd11187.py", line 27, in generate_random_graph
    return {i: sorted(j for j in edges if j > i) for i in range(n)}
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7cd11187.py", line 27, in <genexpr>
    return {i: sorted(j for j in edges if j > i) for i in range(n)}
                  ~~~~~
TypeError: '>' not supported between instances of 'tuple' and 'int'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution, which prevents us from verifying the conjecture's conditions. | next: Investigate and fix the error in the test code to allow for a proper verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14795
2	preregistration	ollama_remote	glm4:latest	0	0	10791
3	novelty	ollama_remote	glm4:latest	0	0	11370
4	novelty	ollama_remote	glm4:latest	0	0	9686
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18034
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14560
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18916
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15682
9	judge	ollama_remote	glm4:latest	0	0	17207

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 131041 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/77cec2684e71.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/77cec2684e71.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/77cec2684e71.tar.gz (if generated)